

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	TARUNIKA S	Reg. No.:	192511024
Date:	29/08/2026	Duration:	60 minutes

Code of Conduct

I TARUNIKA S (192511024) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: TARUNIKA S

1. Problem Analysis & CI/CD Requirements

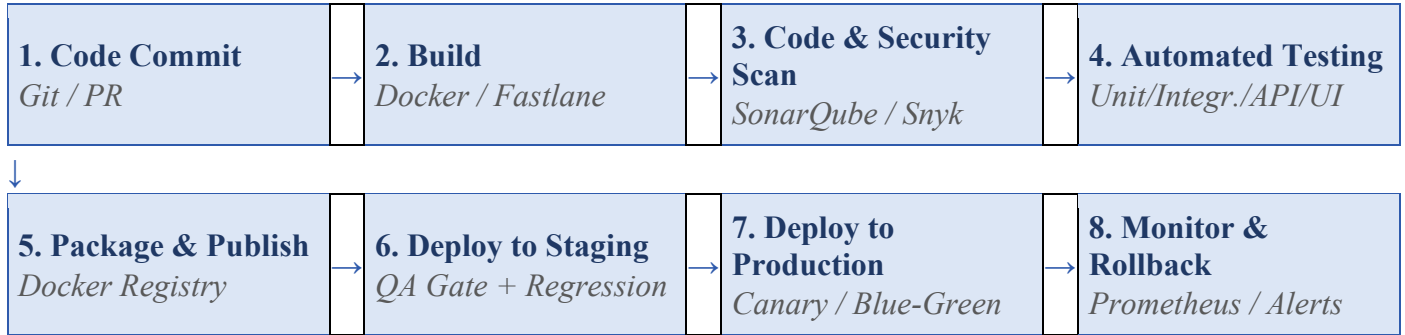
The Smart High-Speed Rail Platform provides e-ticketing, PNR management, and live train tracking. It offers seat information, food, entertainment, Wi-Fi, and station navigation. It supports multiple languages, payments, and delay or platform-change notifications. Its microservices backend connects with existing railway signalling and scheduling systems.

Key CI/CD Drivers Identified

- Frequent, independent releases across many microservices and two mobile platforms plus a web app.
- 24×7 availability — passengers rely on live tracking and ticketing at all hours, so zero/near-zero downtime deployment is mandatory.
- Sensitive payment data mandates PCI-DSS-aligned security scanning and secrets management on every build.
- Sharp traffic spikes during festivals/holidays require automated performance and load testing before release.
- Safety-relevant, real-time data (train position, delays) requires fast, reliable rollback if a release misbehaves.

2. CI/CD Pipeline Architecture & Workflow

The pipeline is triggered on every commit/pull-request and automatically carries validated code from commit to production through eight sequential stages, with human approval gates before Staging and Production.



3. Tools & Technology Selection

Pipeline Stage	Tools / Technology	Justification
Source Control	GitHub / GitLab, GitFlow	Branch protection, mandatory code review and PR-based traceability for many parallel microservice teams.
CI/CD Orchestration	Jenkins / GitHub Actions	Mature plugin ecosystem, parallel job execution across microservices and mobile pipelines.
Build & Containerisation	Docker, Gradle/Maven, Fastlane	Consistent, reproducible builds for backend services and native mobile binaries.
Code Quality (SAST)	SonarQube	Enforces coverage, code-smell and duplication quality gates before promotion.
Dynamic Security (DAST)	OWASP ZAP	Runtime vulnerability scanning of APIs handling PNR and payment data.
Automated Testing	JUnit/TestNG, Selenium, Appium, Postman/Newman, JMeter	Covers unit, web UI, mobile UI, API and load/performance testing respectively.
Artifact Repository	JFrog Artifactory / Docker Hub	Central, versioned storage for container images and mobile build artifacts.

4. Automated Testing Strategy

- **Unit Testing:** Executed on every commit for each microservice and mobile module; build fails if coverage drops below an 80% SonarQube threshold.
- **Integration Testing:** Validates inter-service communication (Ticketing ↔ Payment ↔ Notification) and database interactions in an isolated test environment.
- **API Testing:** Postman/Newman collections run in-pipeline against every service contract, including PNR lookup and live-tracking endpoints.

5. Deployment Strategy Across Environments

- **Development:** Every successful merge to develop auto-deploys to a shared Dev cluster for early integration feedback.
- **Staging / QA:** Deployment requires all automated gates to pass; mirrors Production configuration for full regression, performance and UAT sign-off with manual approval.
- **Production:** Kubernetes-based canary release — 5% → 25% → 100% traffic shift — with automatic health checks at each step; blue-green is used for the payment service to guarantee zero downtime.
- **Feature Flags:** New passenger-facing features (e.g., new tracking UI) are rolled out behind flags for controlled, reversible exposure.

6. Security, Quality Gates, Notifications & Rollback

- **Security Practices:** Secrets (API keys, payment credentials) are stored in HashiCorp Vault/AWS Secrets Manager, never in code; every image is scanned before push and PCI-DSS checks gate the Payment service pipeline.
- **Quality Gates:** A build cannot advance past the Testing stage unless SonarQube quality gate, security scans and the full test suite all pass.
- **Rollback Mechanism:** Prometheus/Grafana monitors error rate and latency post-deployment; if thresholds are breached during canary rollout, the pipeline auto-aborts and Helm performs an automatic rollback to the last stable release, with database migrations reverted via version-controlled rollback scripts.

Key Monitoring KPIs

Metric	Target / Threshold	Action on Breach
API Error Rate	< 1% over 5 minutes	Auto-abort canary; trigger Helm rollback.
P95 Latency (Tracking API)	< 500 ms	Alert on-call via Slack/PagerDuty; investigate autoscaling.
Build/Test Pipeline Success Rate	> 95%	Weekly review; flag flaky tests for remediation.
Container Vulnerability Findings	Zero critical/high	Block image promotion until patched.

7. Environment Configuration Overview

Environment	Configuration	Purpose
Development	Shared K8s namespace, mocked payment gateway, synthetic PNR data, 1 replica/service	Fast feedback for developers on every merge to develop.
Staging / QA	Production-parity K8s cluster, sandbox payment gateway, anonymised production-like data, autoscaled 2-3 replicas	Full regression, performance/load testing and UAT sign-off before release approval.
Production	Multi-zone K8s cluster (EKS/AKS), live payment gateway, autoscaling 3-N replicas, canary + blue-green enabled	Serves live passengers; protected by health checks, monitoring and automated rollback.

Conclusion: This eight-stage, gate-driven CI/CD pipeline gives the Smart High-Speed Rail Passenger Experience Platform fast, safe and independently deployable releases across its microservices and mobile/web clients, while embedding the security, quality and rollback controls required for a safety- and payment-sensitive, always-on passenger system.